KK (Keeney Kode) - Formal Specification

Version: 1.0
Author: John A Keeney - Entrouter - Australia
Date: 2026
Reference implementation: `kk-crypto` v0.1.0 (Rust 2021)

Table of Contents

---

1. Introduction & Notation

1.1 Overview

KK (Keeney Kode) is a novel symmetric cryptographic system where every cryptographic operation - hashing, key derivation, message authentication, encryption, and key agreement - is built from a single primitive: the KK permutation. The permutation operates on a 1600-bit state using two novel building blocks: **Multiply-Fold-Rotate (MFR)** and **Data-Dependent Rotation (DDR)**.

The defining innovation of KK is **temporal permutation variance**: the rotation schedule inside the permutation can be derived from an entropy snapshot, meaning the *mathematical structure* of the cipher changes with every encryption. This is not merely different data through the same algorithm-it is a *different algorithm entirely* at each moment.

1.2 Notation

Symbol	Meaning
$\times_{64}$	Wrapping (modular) 64-bit multiplication
$\oplus$	Bitwise XOR
$\lll r$	Left rotation by r bits on a 64-bit word

$\gg r$	Logical right shift by r bits
$\mathbin{\|}$	Bitwise OR
$\mathbin{&}$	Bitwise AND
$\parallel$	Concatenation of byte strings
$\text{LE}_{k}(x)$	Little-endian encoding of x as k bytes
$S[i]$	Word i of the 25-word state ($0 \le i \le 24$)
ε	An entropy snapshot (32 bytes of mixed entropy + 128-bit nanosecond timestamp)
R	Number of permutation rounds ($R = 32$ default, $R = 20$ for KDF squeeze)
r	Rate in bytes ($r = 152$)
c	Capacity in bytes ($c = 48$)

All arithmetic on 64-bit words is modular ($\bmod 2^{64}$). All multi-byte integers are little-endian unless stated otherwise.

1.3 Security Model

KK assumes a pre-shared secret between sender and receiver. The attacker may observe, replay, or modify ciphertext in transit but does not know the shared secret. KK provides:

- **Confidentiality** via entropy-derived keystream XOR
- **Integrity** via KK-MAC temporal commitment
- **Forward secrecy** via the Rope Ratchet (optional)
- **Mutual authentication** via KK-EKA key agreement (optional)

1.4 Code Reference Convention

Each algorithm section references the implementing function in the `kk-crypto` crate using the notation `→ module::function()`.

---

2. Constants

2.1 State Dimensions

Constant	Value	Description
`STATE_WORDS`	25	64-bit words in the 5×5 state grid
`STATE_BYTES`	200	State size: $25 \times 8 = 200$ bytes (1600 bits)
`ROUNDS`	32	Full permutation rounds
`KDF_SQUEEZE_ROUNDS`	20	Reduced rounds for KDF squeeze
`RATE_WORDS`	19	Sponge rate: 19 words (1216 bits)
`RATE_BYTES`	152	Sponge rate: $19 \times 8 = 152$ bytes

`CAPACITY_WORDS`	6	Sponge capacity: $25 - 19 = 6$ words (384 bits)
`CHUNK_SIZE`	4096	Codec encryption chunk size in bytes

→ `kk_mix.rs` lines 80–102, `codec.rs` line 54

2.2 Domain Separation Bytes

Constant	Value	Usage
`DOMAIN_HASH`	`0x01`	KK-Hash finalization
`DOMAIN_KDF`	`0x02`	KK-KDF finalization
`DOMAIN_MAC`	`0x03`	KK-MAC finalization

→ `kk_mix.rs` lines 119–123

2.3 Initialization Vector (KK_IV)

The 25-word initialization vector is derived as $\lfloor \text{frac}(\sqrt{p_i}) \times 2^{64} \rfloor$ for the first 25 primes $p_1 = 2, p_2 = 3, \ldots, p_{25} = 97$:

Index	Prime	Value
0	$\sqrt{2}$	`0x6A09E667F3BCC908`
1	$\sqrt{3}$	`0xBB67AE8584CAA73B`
2	$\sqrt{5}$	`0x3C6EF372FE94F82B`
3	$\sqrt{7}$	`0xA54FF53A5F1D36F1`
4	$\sqrt{11}$	`0x510E527FADE682D1`
5	$\sqrt{13}$	`0x9B05688C2B3E6C1F`
6	$\sqrt{17}$	`0x1F83D9ABFB41BD6B`
7	$\sqrt{19}$	`0x5BE0CD19137E2179`
8	$\sqrt{23}$	`0xCBBB9D5DC1059ED8`
9	$\sqrt{29}$	`0x629A292A367CD507`
10	$\sqrt{31}$	`0x9159015A3070DD17`
11	$\sqrt{37}$	`0x152FECD8F70E5939`
12	$\sqrt{41}$	`0x67332667FFC00B31`
13	$\sqrt{43}$	`0x8EB44A8768581511`
14	$\sqrt{47}$	`0xDB0C2E0D64F98FA7`
15	$\sqrt{53}$	`0x47B5481DBEFA4FA4`

16	$\sqrt{59}$	`0xAE5F9156E7B6D99B`
17	$\sqrt{61}$	`0xCF6C85D39D1A1E15`
18	$\sqrt{67}$	`0x2F73477D6A4563CA`
19	$\sqrt{71}$	`0x6D1826CAFD82E1ED`
20	$\sqrt{73}$	`0x8B43D4570A51B936`
21	$\sqrt{79}$	`0xE360B596DC380C3F`
22	$\sqrt{83}$	`0x1C456002CE13E9F8`
23	$\sqrt{89}$	`0x6F19633143A0AF0E`
24	$\sqrt{97}$	`0xD94EBEB1AB313933`

These are "nothing up my sleeve" constants; anyone can verify them independently.

→ `kk_mix.rs` lines 128–156

2.4 Default Rotation Schedule

15 pairs of rotation distances, one pair per quintet-round. Each pair contains one value in $[1, 31]$ and one in $[33, 63]$, all odd (coprime with 64, maximizing bit coverage). No value repeats across all 30 entries.

Phase	Quintet	rot_0	rot_1
Row 0	0	7	41
Row 1	1	13	29
Row 2	2	19	37
Row 3	3	23	43
Row 4	4	3	53
Column 0	5	11	47
Column 1	6	17	39
Column 2	7	5	59
Column 3	8	31	49
Column 4	9	9	51
Diagonal 0	10	15	33
Diagonal 1	11	21	45
Diagonal 2	12	27	35
Diagonal 3	13	1	57
Diagonal 4	14	25	55

→ `kk_mix.rs` lines 109–123

2.5 Diagonal Index Patterns

The 5×5 grid (row-major indices 0–24) has 5 wrap-around diagonals:

Diagonal	Indices
0	0, 6, 12, 18, 24
1	1, 7, 13, 19, 20
2	2, 8, 14, 15, 21
3	3, 9, 10, 16, 22
4	4, 5, 11, 17, 23

→ `kk_mix.rs` lines 157–166

2.6 Round Constant Multipliers

Five positions in the 5×5 grid (corners + center) receive round constant injections. Each position has a multiplier:

Position	Grid Location	Multiplier
$S[0]$	top-left	1 (identity)
$S[4]$	top-right	`0x9E3779B97F4A7C15` ($\approx \varphi^{-1} \times 2^{64}$)
$S[12]$	center	`0xB7E151628AED2A6A` ($\approx e^{-1} \times 2^{64}$)
$S[20]$	bottom-left	`0x243F6A8885A2F7A4` ($\approx \pi^{-1} \times 2^{64}$)
$S[24]$	bottom-right	`0x298B075B4B6A5240`

→ `kk_mix.rs` lines 325–329

2.7 Session Domain Labels

Constant	Value	Usage
`DOMAIN_SESSION`	`b"KK-rope-mix-v1"`	Rope Ratchet sponge mix
`STRAND_ENT_INFO`	`b"KK-rope-ent-v1"`	Entropy strand KDF info
`STRAND_TMP_INFO`	`b"KK-rope-tmp-v1"`	Temporal strand KDF info
`STRAND_CHN_INFO`	`b"KK-rope-chn-v1"`	Chain strand KDF info
`INIT_ENT_INFO`	`b"KK-rope-init-ent"`	Initial entropy strand derivation
`INIT_TMP_INFO`	`b"KK-rope-init-tmp"`	Initial temporal strand derivation
`INIT_CHN_INFO`	`b"KK-rope-init-chn"`	Initial chain strand derivation
`EKA_SESSION_INFO`	`b"KK-EKA-session"`	EKA session key derivation

→ `session.rs` lines 67–77, `eka.rs` line 53

---

3. Primitive Operations: MFR, DDR, QuintetRound

3.1 Multiply-Fold-Rotate (MFR)

Definition. For 64-bit words a, b and rotation distance $\text{rot} \in [1, 63]$:

$$\text{MFR}(a, b, \text{rot}) = \big((a \times_{64} (b \mathbin{|} 1)) \oplus ((a \times_{64} (b \mathbin{|} 1)) \gg 32)\big) \lll \text{rot}$$

Equivalently, in three steps:

1. **Multiply:** $\text{product} = a \times_{64} (b \mathbin{|} 1)$
   The OR with 1 forces b odd, guaranteeing the multiplication is bijective (odd numbers are invertible $\bmod 2^{64}$).

2. **Fold:** $\text{folded} = \text{product} \oplus (\text{product} \gg 32)$
   XORing the high 32 bits into the low 32 bits breaks the multiplicative ring structure.

3. **Rotate:** $\text{result} = \text{folded} \lll \text{rot}$
   Fixed-distance rotation spreads the mixed bits across the word.

Properties:

- Non-linear (modular multiplication)
- Bijective in a for fixed b (odd multiplier)
- Full-word mixing through fold and rotate

→ `kk_mix::mfr()` at line 180

3.2 Data-Dependent Rotation (DDR)

Definition. For 64-bit words a, b:

$$\text{DDR}(a, b) = a \lll s$$

where the rotation distance s is computed:

$$\text{folded} = b \oplus (b \gg 32)$$ $$s = \big(\text{folded} \oplus (\text{folded} \gg 16) \oplus (\text{folded} \gg 8)\big) \mathbin{&} 63$$

The folding step ensures that *all* 64 bits of b contribute to the rotation distance, not just the low 6 bits. This is critical for diffusion: without folding, a difference confined to higher bytes of b would be invisible to DDR.

Constant-time implementation. The variable rotation by $s \in [0, 63]$ is decomposed into 6 fixed-distance rotations (by $2^0, 2^1, 2^2, 2^3, 2^4, 2^5$), each conditionally applied via a branchless bitmask:

$$\text{For } i = 0, 1, \ldots, 5:$$ $$m_i = 0 - \big((s \gg i) \mathbin{&} 1\big) \quad (\text{all-zeros or all-ones mask})$$ $$v = (v \mathbin{&} \lnot m_i) \mathbin{|} ((v \lll 2^i) \mathbin{&} m_i)$$

All 6 steps execute unconditionally; no data-dependent branches or variable-distance shifts. Timing is identical regardless of s on all architectures.

Cryptanalytic impact. DDR forces any differential trail to account for all 64 possible rotation distances simultaneously, causing exponential path explosion. No published analysis framework efficiently handles DDR.

→ `kk_mix::ddr()` at line 209

3.3 QuintetRound

Definition. Given five 64-bit words (a, b, c, d, e) and rotation pair $(\text{rot}_0, \text{rot}_1)$:

$$a \leftarrow \text{MFR}(a, b, \text{rot}_0)$$ $$c \leftarrow c \oplus a$$ $$d \leftarrow \text{DDR}(d, c)$$ $$e \leftarrow \text{MFR}(e, d, \text{rot}_1)$$ $$b \leftarrow b \oplus e$$

After one quintet-round, all five words have influenced each other through a chain of non-linear (MFR), linear (XOR), and data-dependent (DDR) operations. No published cipher uses 5-word mixing rounds.

→ `kk_mix::quintet_round()` at line 254

---

4. KK Permutation

4.1 Structure

The KK permutation transforms a 1600-bit state $S = (S[0], S[1], \ldots, S[24])$ over R rounds. Each round consists of:

1. **Row phase** - 5 quintet-rounds on rows of the 5×5 grid
2. **Column phase** - 5 quintet-rounds on columns
3. **Diagonal phase** - 5 quintet-rounds on diagonals
4. **Round constant injection** - at corners + center
5. **Intra-round re-keying** - every 8th round

4.2 Row Phase

For each row $\text{row} \in {0, 1, 2, 3, 4}$, with base index $\text{base} = \text{row} \times 5$:

$$\text{QuintetRound}\big(S[\text{base}],, S[\text{base}+1],, S[\text{base}+2],, S[\text{base}+3],, S[\text{base}+4],, \text{rotations}[\text{row}]\big)$$

4.3 Column Phase

For each column $\text{col} \in {0, 1, 2, 3, 4}$:

$$\text{QuintetRound}\big(S[\text{col}],, S[\text{col}+5],, S[\text{col}+10],, S[\text{col}+15],, S[\text{col}+20],, \text{rotations}[5 + \text{col}]\big)$$

4.4 Diagonal Phase

For each diagonal $d \in {0, 1, 2, 3, 4}$, using the diagonal index patterns from §2.5:

$$\text{Let } (i_0, i_1, i_2, i_3, i_4) = \text{DIAGS}[d]$$ $$\text{QuintetRound}\big(S[i_0],, S[i_1],, S[i_2],, S[i_3],, S[i_4],, \text{rotations}[10 + d]\big)$$

4.5 Round Constant Injection

After the three quintet phases, round constants are injected (wrapping addition) at five positions using the round index $\text{rnd} \in {0, 1, \ldots, R-1}$:

$$S[0] \mathrel{+}= \text{rnd}$$ $$S[4] \mathrel{+}= \text{rnd} \times_{64} \texttt{0x9E3779B97F4A7C15}$$ $$S[12] \mathrel{+}= \text{rnd} \times_{64} \texttt{0xB7E151628AED2A6A}$$ $$S[20] \mathrel{+}= \text{rnd} \times_{64} \texttt{0x243F6A8885A2F7A4}$$ $$S[24] \mathrel{+}= \text{rnd} \times_{64} \texttt{0x298B075B4B6A5240}$$

Note: $\text{rnd} = 0$ produces zero constants for round 0 (all injections are $+0$). Round constants break symmetry and prevent slide attacks.

4.6 Intra-Round Re-Keying

Every 8th round (when $\text{rnd} \bmod 8 = 7$), capacity words are mixed back into rate words:

$$\text{For } i = 0, 1, \ldots, \text{RATE_WORDS}-1:$$ $$S[i] \mathrel{\oplus}= S[\text{RATE_WORDS} + (i \bmod \text{CAPACITY_WORDS})] \lll \text{rnd}$$

This breaks fixed-structure analysis within a single permutation call by feeding the capacity (secret) portion back into the rate (public) portion with round-dependent rotation.

4.7 Computational Cost Per Permutation

Per round: 15 quintet-rounds $= 30$ MFR $+ 15$ DDR $+ 10$ XOR.
Per full permutation ($R = 32$): 480 quintet-rounds $= 960$ MFR $+ 480$ DDR $+ 320$ XOR $+ 160$ wrapping-add (round constants) $+ 4 \times 19 = 76$ re-keying XORs.

→ `kk_mix::kk_permute_n()` at line 279

---

5. Entropy-Derived Rotations

5.1 Entropy Snapshot

An `EntropySnapshot` ε consists of:

Field	Size	Source
`bytes`	32 bytes	Mixed entropy from 4 sources
`timestamp_nanos`	16 bytes (u128 LE)	System time in nanoseconds

Total serialized size: 48 bytes.

The 4 entropy sources, mixed through `kk_entropy_mix()`:

1. **CSPRNG** - 32 bytes from the OS random number generator (`OsRng`)
2. **Timestamp** - System time nanoseconds since epoch
3. **CPU counter** - `RDTSC` XOR'd with stack address (x86_64), or `Instant` fallback
4. **Thread jitter** - 64 measurements of `yield_now()` timing with `black_box` , mixed through `kk_hash`

→ `entropy.rs`

5.2 Rotation Derivation

Given entropy bytes $e_0, e_1, \ldots$, derive 15 rotation pairs:

$$\text{For } i = 0, 1, \ldots, 14 \text{ and } j \in \{0, 1\}:$$ $$\text{idx} = i \times 2 + j$$ $$\text{rotations}[i][j] = (e_{\text{idx}} \mathbin{\&} 63) \mathbin{|} 1$$

Properties:

- $\mathbin{\&} 63$ masks to 6 bits, range $[0, 63]$. Since $256 / 64 = 4$ exactly, there is zero modular bias.
- $\mathbin{|} 1$ forces odd, guaranteeing a non-zero rotation in range $[1, 63]$.
- Requires at least 30 entropy bytes (the first 30 bytes of any entropy source).

- Remaining entries beyond available entropy retain their `DEFAULT_ROTATIONS` values.

Significance. When used with `KkSponge::with_entropy_rotations()`, the algebraic structure of every subsequent permutation call changes. The cipher that processes the data has never existed before and will never exist again.

→ `kk_mix::rotations_from_entropy()` at line 366

5.3 Entropy Mixing

Given n byte-string sources $s_0, s_1, \ldots, s_{n-1}$ and desired output length ℓ:

$$\text{kk_entropy_mix}({s_i}, \ell):$$

1. Sponge $\leftarrow$ `KkSponge::new()`
2. For each source s_i (in order):
   - Absorb $\text{LE}_8(i)$ (source index, 8 bytes)
   - Absorb $\text{LE}_8(|s_i|)$ (source length, 8 bytes)
   - Absorb s_i
3. `finalize_absorb(DOMAIN_HASH)`
4. Return `squeeze( $\ell$ )`

→ `kk_mix::kk_entropy_mix()` at line 815

---

6. KK Sponge

6.1 State

The sponge state consists of:

- S: a 25-word (1600-bit) KK state, initialized to KK_IV
- rotations: a 15×2 rotation schedule (default or entropy-derived)
- buf_pos: byte offset within the current rate block ($0 \le \text{buf_pos} < r$)

6.2 Initialization

Standard: $S \leftarrow \text{KK_IV}$, $\text{rotations} \leftarrow \text{DEFAULT_ROTATIONS}$, $\text{buf_pos} \leftarrow 0$.

With entropy rotations: Same, but $\text{rotations} \leftarrow \text{rotations_from_entropy}(\text{entropy})$.

→ `kk_mix::KkSponge::new()`, `KkSponge::with_entropy_rotations()`

6.3 Absorb

Input: byte string $M = m_0 m_1 \ldots m_{n-1}$.

The absorb operation XORs input bytes into the rate portion of the state, permuting after every $r = 152$ bytes:

1. For each byte m_k of M:
   - XOR m_k into S at rate position buf_pos (byte-level addressing into the first 19 words, little-endian):
     - Word index: $\text{buf_pos} / 8$
     - Bit shift: $(\text{buf_pos} \bmod 8) \times 8$
   - $\text{buf_pos} \leftarrow \text{buf_pos} + 1$

- If $\text{buf_pos} = r$: permute S, set $\text{buf_pos} \leftarrow 0$

Optimization: When buf_pos is word-aligned and ≥ 8 bytes remain, full 64-bit words are XOR'd directly, reducing operations by $8\times$.

→ `kk_mix::KkSponge::absorb()` at line 462

6.4 Finalize Absorb (Domain-Separated Padding)

Input: domain separation byte $d \in \{$ `0x01` , `0x02` , `0x03` $\}$.

Multi-rate padding:

1. XOR d into S at rate position buf_pos
2. XOR `0x80` into S at rate position $r - 1$ (last byte of rate)
3. Permute S
4. Set $\text{buf_pos} \leftarrow 0$

This ensures:

- Domain separation between hash, KDF, and MAC modes
- Injective padding (no two messages produce the same padded state)
- The final permutation fully mixes the domain byte and padding

→ `kk_mix::KkSponge::finalize_absorb()` at line 506

6.5 Squeeze

Input: desired output length ℓ bytes.

1. Read up to r bytes from the rate portion of S (starting from $\text{buf_pos} = 0$)
2. If more bytes needed: permute S (using $R = 32$ rounds), read next r-byte block
3. Repeat until ℓ bytes produced
4. Return the first ℓ bytes

→ `kk_mix::KkSponge::squeeze()`

6.6 Squeeze KDF

Identical to Squeeze (§6.5) but uses $R = 20$ rounds (`KDF_SQUEEZE_ROUNDS`) between blocks instead of $R = 32$. The reduced round count is secure because each squeeze block operates on a keyed, domain-separated state that the attacker cannot observe or influence directly.

→ `kk_mix::KkSponge::squeeze_kdf()`

---

7. KK-Hash, KK-KDF, KK-MAC

7.1 KK-Hash

Input: byte string M.
Output: 32-byte digest.

$$\text{KK-Hash}(M):$$

1. Sponge $\leftarrow$ `KkSponge::new()`
2. Absorb M

3. `finalize_absorb(DOMAIN_HASH)` (domain byte `0x01`)
4. Return `squeeze(32)`

→ `kk_mix::kk_hash()`

7.2 KK-KDF (Key Derivation Function)

Input: key K, salt σ, info I, output length ℓ.
Output: ℓ-byte derived key.

$$\text{KK-KDF}(K, \sigma, I, \ell):$$

1. Sponge $\leftarrow$ `KkSponge::with_entropy_rotations(` σ `)`
2. Absorb K
3. Absorb $\text{LE}_8(|\sigma|) \parallel \sigma$ (length-prefixed salt)
4. Absorb $\text{LE}_8(|I|) \parallel I$ (length-prefixed info)
5. `finalize_absorb(DOMAIN_KDF)` (domain byte `0x02`)
6. Return `squeeze_kdf(` ℓ `)` (uses 20-round squeeze)

Key properties:

- The salt determines the rotation schedule, making the permutation structure salt-dependent
- Length-prefixed inputs prevent canonicalization attacks (e.g., salt `"ab"` + info `"cd"` vs salt `"abc"` + info `"d"`)
- Domain byte `0x02` separates KDF from hash/MAC

→ `kk_mix::kk_kdf()`

7.3 KK-KDF Batch (8-lane)

Input: key K, salt σ, 8 info strings $I_0, \ldots, I_7$, output length ℓ.
Output: 8 derived keys, each ℓ bytes.

1. Construct a shared sponge prefix: absorb K, then $\text{LE}_8(|\sigma|) \parallel \sigma$
2. Clone the sponge 8 times
3. Each clone i absorbs $\text{LE}_8(|I_i|) \parallel I_i$, then `finalize_absorb(DOMAIN_KDF)`
4. Squeeze all 8 in parallel:
   - **x86_64 with AVX-512:** Pack 8 sponge states into 25 SIMD registers (`__m512i`), perform the permutation 8-wide in a single pass
   - **Fallback:** Sequential scalar squeeze for each clone

→ `kk_mix::kk_kdf_batch_8()`

7.4 KK-MAC (Message Authentication Code)

Input: key K, message M.
Output: 32-byte authentication tag.

$$\text{KK-MAC}(K, M):$$

1. Sponge $\leftarrow$ `KkSponge::new()`
2. Absorb $\text{LE}_8(|K|) \parallel K$ (length-prefixed key prevents length-extension)
3. Absorb M
4. `finalize_absorb(DOMAIN_MAC)` (domain byte `0x03`)
5. Return `squeeze(32)`

6. Zeroize intermediate squeeze output

Deterministic: same (K, M) always produces the same tag. Protocols requiring unique tags must prepend a nonce to M.

→ `kk_mix::kk_mac()`

7.5 KK-MAC Verify

Input: key K, message M, expected tag T.
Output: boolean.

1. Compute $T' = \text{KK-MAC}(K, M)$
2. Return `constant_time_eq( $T'$ , $T$ )`

Constant-time comparison: accumulate $\text{diff} = \bigoplus_{i=0}^{31} (T'[i] \oplus T[i])$, pass through `black_box()`, return $\text{diff} = 0$.

→ `kk_mix::kk_mac_verify()`

7.6 KK-MAC with Entropy-Derived Rotations

Input: key K, message M, entropy bytes E.
Output: 32-byte authentication tag.

$$\text{KK-MAC-Entropy}(K, M, E):$$

1. Sponge $\leftarrow$ `KkSponge::with_entropy_rotations( $E$ )`
2. Absorb $\text{LE}_8(|K|) \parallel K$
3. Absorb M
4. `finalize_absorb(DOMAIN_MAC)`
5. Return `squeeze(32)`

The permutation's mathematical structure depends on E, so the MAC computation that produced the tag only existed at that entropic moment. Used by the temporal proof system (§9.4).

→ `kk_mix::kk_mac_with_entropy()`

---

8. KK Codec (Stream Cipher Mode)

8.1 Per-Chunk Keystream Derivation

Plaintext is divided into chunks of `CHUNK_SIZE` = 4096 bytes. For chunk index i (0-based):

$$\text{info}_i = \texttt{b"KK-sym-v1\textbackslash 0"} \parallel \text{LE}*8(i) \parallel \text{LE}*{16}(\varepsilon.\text{timestamp_nanos})$$

$$\text{keystream}_i = \text{KK-KDF}(\text{shared_secret},; \varepsilon.\text{bytes},; \text{info}_i,; \text{chunk_len})$$

Each chunk's keystream is derived independently, enabling parallel computation. The entropy snapshot ε serves as the KDF salt, making the permutation structure (rotation schedule) unique per encryption.

→ `kdf::derive_symbol_key()`

8.2 Batch Keystream (8-chunk)

Full batches of 8 consecutive chunks use `kk_kdf_batch_8()` for SIMD acceleration. Each batch of 8 `info` strings shares the same key and salt prefix; only the info (chunk index + timestamp) varies.

Additional parallelism: `rayon` splits the plaintext into groups of $8 \times 4096 = 32768$ bytes, each processed in parallel.

→ `codec::xor_with_keystream()`

8.3 Encryption (XOR with keystream)

$$\text{ciphertext}[i \times 4096 \ldots (i+1) \times 4096] = \text{plaintext}[\ldots] \oplus \text{keystream}_i$$

For the final partial chunk, only the required prefix of the keystream is used. All keystream material is zeroized after XOR.

8.4 Encode

Input: shared secret K, plaintext P.
Output: `KkPacket` .

$$\text{encode}(K, P):$$

1. $\varepsilon \leftarrow \text{entropy::gather()}$
2. $C \leftarrow \text{xor_with_keystream}(K, \varepsilon, P)$
3. $\tau \leftarrow \text{temporal::commit}(K, \varepsilon, C)$
4. Return `KkPacket` ${C, \varepsilon, \tau}$

→ `codec::encode()`

8.5 Decode

Input: shared secret K, `KkPacket` ${C, \varepsilon, \tau}$.
Output: plaintext P or error.

$$\text{decode}(K, {C, \varepsilon, \tau}):$$

1. $\text{temporal::verify}(K, \varepsilon, C, \tau) \rightarrow$ error if mismatch
2. $P \leftarrow \text{xor_with_keystream}(K, \varepsilon, C)$
3. Return P

Verify-before-decrypt: integrity is checked before any plaintext is produced, preventing partial plaintext leaks.

→ `codec::decode()`

8.6 Split-Channel Mode

For protocols that transmit the entropy snapshot ε on a separate channel:

- `encode_split(K, P)` → $(\varepsilon, \text{KkSealedMessage}{C, \tau})$
- `decode_split(K, sealed, \varepsilon)` → P

`KkSealedMessage` omits the 48-byte snapshot, carrying only ciphertext + commitment.

→ `codec::encode_split()` , `codec::decode_split()`

8.7 Split-Channel Empirical Verification

The `examples/split_demo.rs` program exercises the full split-channel pipeline:

Test	Measured Value
Public channel payload	98 bytes (ciphertext + commitment)
Private channel payload	48 bytes (entropy snapshot)
Public-only decode attempt	UNBREAKABLE (fails without entropy)
Tampered sealed message decode	REJECTED (commitment mismatch)
Legitimate split decode	SUCCESS (plaintext recovered)

The three attack scenarios confirm that possession of the public channel alone reveals nothing, tampered messages are detected via the temporal commitment, and only a receiver with both channels can recover plaintext.

→ `examples/split_demo.rs`

---

9. Temporal Commitment

9.1 Commitment Key Derivation

$$\text{commit_key} = \text{KK-KDF}(K, \varepsilon.\text{bytes}, \texttt{b"KK-commit-v1"}, 32)$$

→ `kdf::derive_commitment_key()`

9.2 Commit

Input: shared secret K, entropy ε, ciphertext C.
Output: 32-byte `TemporalCommitment`.

1. $\text{ck} \leftarrow \text{derive_commitment_key}(K, \varepsilon)$
2. $\text{msg} \leftarrow \varepsilon.\text{bytes} \parallel \text{LE}_{16}(\varepsilon.\text{timestamp_nanos}) \parallel C$
3. $\text{mac} \leftarrow \text{KK-MAC}(\text{ck}, \text{msg})$
4. Zeroize ck
5. Return `TemporalCommitment` ${\text{mac}}$

→ `temporal::commit()`

9.3 Verify

Input: shared secret K, entropy ε, ciphertext C, expected commitment τ.
Output: `Ok(())` or `Err(CommitmentMismatch)`.

1. Re-derive ck and msg as in Commit
2. `kk_mac_verify(ck, msg, τ.mac)` → error if false

→ `temporal::verify()`

9.4 Bound Commitment (Challenge-Response)

For protocols requiring freshness guarantees beyond the basic commitment.

TemporalProof structure (96 bytes):

Field	Size	Description
`mac`	32 bytes	MAC tag
`nonce`	32 bytes	Challenge nonce
`prev_mac`	32 bytes	Previous MAC in chain

Genesis: $\text{prev_mac} = [0; 32]$ (all zeros) for the first message in a chain.

9.4.1 Generate Challenge

$$\text{nonce} \leftarrow \text{OsRng}(32)$$

→ `temporal::generate_challenge()`

9.4.2 Commit Bound

Input: shared secret K, entropy ε, ciphertext C, nonce N, previous MAC prev.
Output: 96-byte `TemporalProof`.

1. $\text{ck} \leftarrow \text{derive_commitment_key}(K, \varepsilon)$
2. $\text{msg} \leftarrow N \parallel \text{prev} \parallel \varepsilon.\text{bytes} \parallel \text{LE}_{16}(\varepsilon.\text{timestamp_nanos}) \parallel C$
3. $\text{mac} \leftarrow \text{KK-MAC-Entropy}(\text{ck}, \text{msg}, \varepsilon.\text{bytes})$
   - note: uses entropy-derived rotations for the MAC itself
4. Return `TemporalProof` ${\text{mac}, N, \text{prev}}$

→ `temporal::commit_bound()`

9.4.3 Verify Bound

Input: shared secret K, entropy ε, ciphertext C, proof π, expected nonce N_{exp}, expected previous MAC prev_{exp}, max epoch drift Δ.
Output: `Ok(())` or error.

Three-step verification:

1. **Nonce check:** $\pi.\text{nonce} = N_{\text{exp}}$ → `StaleNonce` error if mismatch
2. **Epoch drift:** $|\text{now} - \varepsilon.\text{timestamp_nanos}| \le \Delta$ → `EpochDrift` error if exceeded
3. **MAC verify:** Recompute as in §9.4.2 using `kk_mac_verify_with_entropy()` → `CommitmentMismatch` error if mismatch

The caller is responsible for:

- Tracking nonces (each nonce should be used exactly once)
- Maintaining the `prev_mac` chain for sequential ordering

→ `temporal::verify_bound()`

9.5 Commitment Binding Tests

Integration tests verify that the temporal commitment rejects every category of tampering:

Tampering Scenario	Expected Result	Verified

Flip any bit in ciphertext	`CommitmentMismatch` error	Yes
Modify `timestamp_nanos` in entropy snapshot	`CommitmentMismatch` error	Yes
Substitute different entropy bytes	`CommitmentMismatch` error	Yes
Bound commitment with wrong `prev_mac`	Chain integrity failure	Yes
Bound commitment with wrong nonce	`StaleNonce` error	Yes

All tampering scenarios produce deterministic, immediate rejection before any plaintext is produced (verify-before-decrypt).

→ `tests/integration.rs` (temporal commitment test suite)

---

10. AEAD Mode

10.1 Overview

KK-AEAD (Authenticated Encryption with Associated Data) extends the basic codec with authenticated-but-unencrypted associated data. The AAD is bound into the temporal commitment but is not XOR'd with keystream.

10.2 AEAD Commitment

$$\text{commit_aead}(K, \varepsilon, C, \text{AAD}):$$

1. $\text{ck} \leftarrow \text{derive_commitment_key}(K, \varepsilon)$
2. $\text{msg} \leftarrow \varepsilon.\text{bytes} \parallel \text{LE}_{16}(\varepsilon.\text{timestamp_nanos}) \parallel \text{LE}_8(|\text{AAD}|) \parallel \text{AAD} \parallel C$
3. $\text{mac} \leftarrow \text{KK-MAC}(\text{ck}, \text{msg})$
4. Return `TemporalCommitment` ${\text{mac}}$

The AAD length is encoded as 8 bytes (LE u64) before the AAD itself, preventing canonicalization between AAD and ciphertext boundaries.

→ `temporal::commit_aead()`

10.3 AEAD Encode

Input: shared secret K, plaintext P, associated data A.
Output: `KkAeadPacket` .

1. $\varepsilon \leftarrow \text{entropy::gather()}$
2. $C \leftarrow \text{xor_with_keystream}(K, \varepsilon, P)$
3. $\tau \leftarrow \text{temporal::commit_aead}(K, \varepsilon, C, A)$
4. Return `KkAeadPacket` ${A, C, \varepsilon, \tau}$

→ `codec::encode_aead()`

10.4 AEAD Decode

Input: shared secret K, `KkAeadPacket` ${A, C, \varepsilon, \tau}$.
Output: plaintext P or error.

1. $\text{temporal::verify_aead}(K, \varepsilon, C, A, \tau) \rightarrow$ error if mismatch
2. $P \leftarrow \text{xor_with_keystream}(K, \varepsilon, C)$

3. Return P

→ `codec::decode_aead()`

---

11. Rope Ratchet (Forward Secrecy)

11.1 Overview

The Rope Ratchet is a 4-strand ratchet providing ~192-bit forward secrecy using only KK primitives. Once a message key is derived and the ratchet advances, the old state is zeroized and irrecoverable.

Strand	Source	Purpose
Entropy	`EntropySnapshot.bytes`	Environmental randomness per message
Temporal	`ε.timestamp_nanos`	Binds ratchet to real-world time
Chain	Previous chain strand	One-way forward secrecy
Counter	Monotonic `u64`	Deterministic ordering

Innovation: All 4 strand outputs are fed into a single KK sponge with entropy-derived rotations, so both the key AND the algebraic structure of the permutation change with every message.

11.2 Initialization

Input: shared secret K, direction context ctx (e.g., `b"alice-to-bob"`).

1. $\sigma \leftarrow \text{KK-Hash}(\text{ctx})$ (32-byte salt)
2. $E_0 \leftarrow \text{KK-KDF}(K, \sigma, \texttt{b"KK-rope-init-ent"}, 32)$
3. $T_0 \leftarrow \text{KK-KDF}(K, \sigma, \texttt{b"KK-rope-init-tmp"}, 32)$
4. $C_0 \leftarrow \text{KK-KDF}(K, \sigma, \texttt{b"KK-rope-init-chn"}, 32)$
5. $\text{counter} \leftarrow 0$

Zeroize intermediate KDF outputs after copying into strand arrays.

→ `session::RopeRatchet::new()`

11.3 Ratchet Step

Input: entropy snapshot ε.
Output: 32-byte message key.

Strand Evolution

1. **Entropy strand:** $$E_{n+1} \leftarrow \text{KK-KDF}(E_n, \varepsilon.\text{bytes}, \texttt{b"KK-rope-ent-v1"}, 32)$$
2. **Temporal strand:** $$T_{n+1} \leftarrow \text{KK-KDF}(T_n, \text{LE}_{16}(\varepsilon.\text{timestamp\_nanos}), \texttt{b"KK-rope-tmp-v1"}, 32)$$
3. **Chain strand** (counter incremented first): $$\text{counter} \leftarrow \text{counter} + 1$$ $$C_{n+1}^{\text{pre}} \leftarrow \text{KK-KDF}(C_n, \text{LE}_8(\text{counter}), \texttt{b"KK-rope-chn-v1"}, 32)$$

Strand Mixing (The KK Innovation)

4. Concatenate all 4 strands: $$\text{combined} = E_{n+1} \parallel T_{n+1} \parallel C_{n+1}^{\text{pre}} \parallel \text{LE}_8(\text{counter}) \quad (104 \text{ bytes})$$
5. Mix through KK-KDF with entropy-derived rotations: $$\text{output} \leftarrow \text{KK-KDF}(\text{combined}, \varepsilon.\text{bytes}, \texttt{b"KK-rope-mix-v1"}, 64)$$
6. Split the 64-byte output:
    - $C_{n+1} \leftarrow \text{output}[0..32]$ (new chain strand - forward secrecy)
    - $\text{message_key} \leftarrow \text{output}[32..64]$ (returned to caller)
7. Zeroize combined and output.

The old chain strand value is overwritten; backward computation is impossible.

→ `session::RopeRatchet::step()`

11.4 RopeStep Metadata

Each ratchet advance produces metadata that must be transmitted alongside the ciphertext so the receiver can reproduce the derivation:

Field	Size	Description
`counter`	8 bytes (u64 LE)	Message sequence number
`snapshot`	48 bytes	Entropy snapshot (§5.1)
Total	**56 bytes**	

→ `session::RopeStep`

11.5 Sender: Advance

1. $\varepsilon \leftarrow \text{entropy::gather()}$
2. $(\text{message_key}, \text{step}) \leftarrow \text{ratchet.step}(\varepsilon)$ with $\text{step} = (\varepsilon, \text{counter})$
3. Return $(\text{message_key}, \text{step})$

→ `session::RopeRatchet::advance()`

11.6 Receiver: Receive

Input: `RopeStep` from sender.

1. Verify $\text{step.counter} = \text{self.counter} + 1$ → error if out of order (strict ordering)
2. $\text{message_key} \leftarrow \text{ratchet.step}(\text{step.snapshot})$
3. Return message_key

→ `session::RopeRatchet::receive()`

11.7 Encode Session

Input: ratchet, plaintext P.
Output: `RopePacket`.

1. $(\text{mk}, \text{step}) \leftarrow \text{ratchet.advance()}$
2. $\text{inner} \leftarrow \text{codec::encode}(\text{mk}, P)$ - inner packet uses its own independent entropy
3. Zeroize mk
4. Return `RopePacket` ${\text{step}, \text{inner}}$

Double entropy: The ratchet step uses one ε for key derivation; the inner `KkPacket` captures its own independent snapshot for per-symbol encryption. Two unrepeatable moments per message.

→ `session::encode_session()`

11.8 Decode Session

Input: ratchet, `RopePacket` ${\text{step}, \text{inner}}$.
Output: plaintext P or error.

1. $\text{mk} \leftarrow \text{ratchet.receive}(\text{step})$
2. $P \leftarrow \text{codec::decode}(\text{mk}, \text{inner})$
3. Zeroize mk
4. Return P

→ `session::decode_session()`

11.9 Session AEAD

`encode_session_aead()` and `decode_session_aead()` combine the Rope Ratchet with AEAD mode. The ratchet derives the message key; the inner packet is a `KkAeadPacket` with AAD authenticated but not encrypted.

→ `session::encode_session_aead()`, `session::decode_session_aead()`

---

12. KK-EKA (Entropy Key Agreement)

KK-EKA is a 3-message, PSK-authenticated key agreement protocol. Two parties who share a pre-shared key (PSK) exchange fresh entropy and derive a session key without ever transmitting the PSK.

→ `eka.rs`

12.1 Overview

Property	Mechanism
Mutual authentication	KK-MAC tags over entropy ‖ peer data, keyed by PSK
Contributory entropy	Session key mixes entropy from **both** parties
Commitment binding	Initiator commits to its entropy before seeing responder's
Forward secrecy	Ephemeral entropy is zeroized after key derivation
Temporal freshness	Each `EntropySnapshot` includes CPU counters and OS randomness

12.2 Wire Types

Message	Size	Layout
`EkaMsg1`	32 B	`commit_a` (32 B)

`EkaMsg2`	80 B	`entropy_b_bytes` (48 B) ‖ `auth_b` (32 B)
`EkaMsg3`	80 B	`entropy_a_bytes` (48 B) ‖ `auth_a` (32 B)

- `commit_a = kk_hash(serialize(ε_a))` - a 32-byte hash of the initiator's entropy snapshot.
- `auth_b = kk_mac(psk, ε_b\_bytes \| commit_a)` - authenticates responder's entropy bound to the initiator's commitment.
- `auth_a = kk_mac(psk, ε_a\_bytes \| ε_b\_bytes)` - authenticates initiator's entropy bound to both snapshots.

12.3 Protocol Flow

```
Initiator (Alice)                    Responder (Bob)
─────────────────                    ─────────────────

ε_a ← gather_entropy()
commit_a ← kk_hash(ε_a.to_bytes())

        ── EkaMsg1 {commit_a} ──▶

                                     ε_b ← gather_entropy()
                                     auth_b ← kk_mac(psk, ε_b ‖ commit_a)

        ◀── EkaMsg2 {ε_b, auth_b} ──

verify auth_b:
  expected ← kk_mac(psk, ε_b ‖ commit_a)
  assert constant_time_eq(auth_b, expected)
auth_a ← kk_mac(psk, ε_a ‖ ε_b)

        ── EkaMsg3 {ε_a, auth_a} ──▶

                                     verify commit:
                                       assert kk_hash(ε_a) == commit_a
                                     verify auth_a:
                                       expected ← kk_mac(psk, ε_a ‖ ε_b)
                                       assert constant_time_eq(auth_a, expected)

Both derive: session_key ← kk_kdf(psk, ε_a ‖ ε_b, "KK-EKA-session", 32)
```

12.4 State Machines

`EkaInitiator`

Method	State transition
`new(psk)`	Gathers $ε_a$; computes commit_a; returns `(Self, EkaMsg1)`
`process_msg2(msg2)`	Verifies auth_b; computes auth_a; derives session key; returns `(session_key, EkaMsg3)`

`EkaResponder`

Method	State transition
`new(psk, msg1)`	Gathers $ε_b$; computes auth_b; stores commit_a; returns `(Self, EkaMsg2)`
`process_msg3(msg3)`	Verifies commit_a (hash match); verifies auth_a; derives session key; returns `session_key`

12.5 Session Key Derivation

$$ \text{session_key} = \text{kk_kdf}(\text{psk},; ε_a_\text{bytes} ,|, ε_b_\text{bytes},; \texttt{"KK-EKA-session"},; 32) $$

The KDF is invoked with:

- **key material:** the PSK
- **salt:** concatenation of both entropy snapshots (96 bytes total)
- **info:** the literal ASCII string `"KK-EKA-session"`
- **output length:** 32 bytes

12.6 Zeroization

Both `EkaInitiator` and `EkaResponder` implement `Drop`. On drop:

- All ephemeral entropy bytes are overwritten with zeros.
- The stored PSK copy is overwritten with zeros.
- Computed MAC values are overwritten with zeros.

This ensures that compromise of memory after the handshake cannot recover the ephemeral material used to derive the session key.

12.7 EKA Empirical Verification

Integration tests exercise the complete KK-EKA protocol flow:

Test	Verified Property
Full 3-message handshake	Both parties derive identical 32-byte session key
Commitment binding	Bob verifies `commit_a == kk_hash(ε_a.to_bytes())`; mismatch rejects
Authentication tags	Modified `auth_b` or `auth_a` causes immediate rejection
Entropy contribution	Session key changes if either party's entropy changes
Zeroization on drop	All ephemeral material overwritten after handshake completion
Session key into Rope Ratchet	Derived key feeds directly into `RopeRatchet::new()`

→ `tests/integration.rs` (EKA test suite), `examples/basic.rs`

---

13. Security Claims

> ***Disclaimer:*** *KK-Crypto is a novel, un-audited cryptographic primitive. It has **not** been reviewed by third-party cryptographers. Do not use for production security until a formal audit has been conducted.*

13.1 Threat Model

KK assumes a **pre-shared key (PSK)** between sender and receiver. The attacker may:

- Observe all ciphertext in transit.
- Replay captured packets.
- Modify or inject ciphertext.
- Know the protocol specification and all public parameters.

The attacker does **not** know the PSK.

13.2 Confidentiality

Each encoding captures a unique `EntropySnapshot` (CPU performance counters, thread scheduling jitter, OS-provided randomness). The snapshot feeds the KK-KDF to derive per-chunk keystream with the info string:

$$ \text{info} = \texttt{b"KK-sym-v1\textbackslash 0"} ,|, \text{LE8}(i) ,|, \text{LE16}(\text{timestamp}) $$

This ensures the same plaintext never produces the same ciphertext twice, even with the same key, because the entropy snapshot (and therefore the keystream) differs every time.

13.3 Integrity

Every `KkPacket` carries a KK-MAC tag over (ciphertext || entropy snapshot). The `decode` function rejects any packet whose tag does not verify, preventing silent tampering. Verification uses constant-time comparison with `black_box` barriers to prevent compiler elision.

13.4 Temporal Binding

The `TemporalCommitment` in each packet commits to the entropy used during encoding:

$$ \text{commitment} = \text{kk_mac}(\text{commit_key},; \varepsilon.\text{bytes} ,|, \text{LE16}(\text{timestamp}) ,|, C) $$

where $\text{commit_key} = \text{kk_kdf}(\text{secret},; \varepsilon.\text{bytes},; \texttt{"KK-commit-v1"},; 32)$.

The receiver re-derives the commitment and rejects packets where it does not match.

13.5 AEAD Binding

In AEAD mode, associated data (AAD) is bound into the commitment:

$$ \text{msg} = \text{LE8}(|\text{AAD}|) ,|, \text{AAD} ,|, C $$

This prevents an attacker from transplanting ciphertext between different AAD contexts.

13.6 Forward Secrecy

The base `codec` module provides **no** forward secrecy - compromise of the long-term PSK exposes all past messages.

The `session` module's **Rope Ratchet** provides ~192-bit forward secrecy:

- 4 strands (entropy, temporal, chain, counter) are irreversibly evolved on every step.
- Strand mixing produces a 64-byte intermediate via KK-Hash, split into a 32-byte message key and a new 32-byte chain strand.
- Old strand values are overwritten and become unrecoverable.
- The 384-bit sponge capacity bounds the security level at ~192 bits.

13.7 Key Hygiene

- Intermediate keys (commit keys, chunk keystream, message keys) are zeroized via the `zeroize` crate immediately after use.
- Output buffers are zeroized on error paths to prevent partial plaintext leaks.
- `EkaInitiator`, `EkaResponder`, and `RopeRatchet` all implement `Drop` with full zeroization.

13.8 Known Limitations

Limitation	Mitigation
Novel, un-audited primitive	Treat as experimental; see §13 header
No replay protection in base codec	Add sequence numbers or timestamps at the protocol layer, or use the Rope Ratchet (monotonic counter)
PSK-only authentication	Use KK-EKA (§12) for ephemeral key agreement
No post-quantum security claim	The permutation is symmetric; no public-key operations to attack with Grover/Shor, but no formal post-quantum analysis
Entropy quality depends on platform	`gather_entropy()` uses OS randomness + CPU counters + jitter; degraded-entropy environments may weaken security

---

14. Wire Format Summary

All multi-byte integers are **little-endian**. All fields are concatenated with no padding or delimiters.

14.1 EntropySnapshot

Offset	Size	Field
0	32 B	`bytes` - environmental entropy
32	16 B	`timestamp` - `u128` nanosecond counter (LE)
Total	**48 B**	

→ `entropy::EntropySnapshot`

14.2 KkPacket

Offset	Size	Field
0	4 B	`ct_len` - ciphertext length (`u32` LE)
4	`ct_len`	ciphertext
4 + ct_len	48 B	`snapshot` - `EntropySnapshot`
52 + ct_len	32 B	`commitment` - `TemporalCommitment`
Total	**84 + ct_len**	

→ `codec::KkPacket`, `codec::encode()` / `codec::decode()`

14.3 KkSealedMessage

Offset	Size	Field
0	4 B	`ct_len` - ciphertext length (`u32` LE)
4	`ct_len`	ciphertext
4 + ct_len	32 B	`commitment` - `TemporalCommitment`
Total	**36 + ct_len**	

→ `codec::KkSealedMessage` , `codec::encode_split()`

14.4 KkBoundPacket

Offset	Size	Field
0	4 B	`ct_len` - ciphertext length (`u32` LE)
4	`ct_len`	ciphertext
4 + ct_len	48 B	`snapshot` - `EntropySnapshot`
52 + ct_len	96 B	`proof` - `TemporalProof`
Total	**148 + ct_len**	

→ `codec::KkBoundPacket` , `codec::encode_bound()`

14.5 TemporalProof

Offset	Size	Field
0	32 B	`mac` - KK-MAC tag
32	32 B	`nonce` - challenge nonce
64	32 B	`prev_mac` - commitment to prior state
Total	**96 B**	

→ `temporal::TemporalProof`

14.6 KkAeadPacket

Offset	Size	Field
0	4 B	`aad_len` - AAD length (`u32` LE)
4	`aad_len`	associated data (authenticated, not encrypted)
4 + aad_len	4 B	`ct_len` - ciphertext length (`u32` LE)
8 + aad_len	`ct_len`	ciphertext
8 + aad_len + ct_len	48 B	`snapshot` - `EntropySnapshot`

56 + aad_len + ct_len	32 B	`commitment` - `TemporalCommitment`
Total	**88 + aad_len + ct_len**	

→ `codec::KkAeadPacket` , `codec::encode_aead()` / `codec::decode_aead()`

14.7 RopeStep

Offset	Size	Field
0	8 B	`counter` - message sequence number (`u64` LE)
8	48 B	`snapshot` - `EntropySnapshot`
Total	**56 B**	

→ `session::RopeStep`

14.8 RopePacket

Offset	Size	Field
0	56 B	`step` - `RopeStep`
56	variable	inner `KkPacket` bytes
Total	**56 + inner_len**	

→ `session::encode_session()` / `session::decode_session()`

14.9 RopeAeadPacket

Offset	Size	Field
0	56 B	`step` - `RopeStep`
56	variable	inner `KkAeadPacket` bytes
Total	**56 + inner_len**	

→ `session::encode_session_aead()` / `session::decode_session_aead()`

14.10 EKA Messages

EkaMsg1 (Initiator → Responder)

Offset	Size	Field
0	32 B	`commit_a` - `kk_hash(ε_a.to_bytes())`
Total	**32 B**	

EkaMsg2 (Responder → Initiator)

Offset	Size	Field

0	48 B	`entropy_b_bytes` - responder's `EntropySnapshot`
48	32 B	`auth_b` - `kk_mac(psk, ε_b ‖ commit_a)`
Total	**80 B**	

EkaMsg3 (Initiator → Responder)

Offset	Size	Field
0	48 B	`entropy_a_bytes` - initiator's `EntropySnapshot`
48	32 B	`auth_a` - `kk_mac(psk, ε_a ‖ ε_b)`
Total	**80 B**	

→ `eka::EkaMsg1` , `eka::EkaMsg2` , `eka::EkaMsg3`

---

15. Test Vector Reference

All test vectors are published in `KK_TEST_VECTORS.md` , generated deterministically by `examples/generate_vectors.rs` and verified by `tests/vectors.rs` (44 tests).

15.1 Canonical Snapshots

Five deterministic snapshots are used across all vectors:

$$ \text{snapshot}_i.\text{bytes}[j] = (i + j) \bmod 256, \quad \text{timestamp}_i = 1{,}000{,}000{,}000 + i \times 111{,}111{,}111 $$

for $i \in {0, 1, 2, 3, 4}$ and $j \in {0, \ldots, 31}$.

15.2 Vector Categories

Category	Count	Description
`kk_hash`	10	Hash of canonical inputs (empty, single byte, repeated patterns, snapshot bytes)
`kk_kdf`	8	Key derivation with varying key material, salt, info, and output lengths
`kk_mac`	6	MAC tags over canonical messages with different keys
`kk_mac_with_entropy`	3	Entropy-augmented MAC using canonical snapshots
`kk_permute`	1+	Full 384-bit permutation output for all-zeros input
`rotations_from_entropy`	-	Entropy-derived rotation schedules
`encode / decode`	-	Codec roundtrip with deterministic snapshots
`encode_aead / decode_aead`	-	AEAD roundtrip with AAD binding
`RopeRatchet`	-	Session ratchet advance/receive consistency

15.3 Verification

To regenerate and verify all vectors:

```
# Generate vectors (writes to docs/KK_TEST_VECTORS.md)
cargo run --example generate_vectors

# Verify vectors (44 tests)
cargo test --test vectors
```

All vector tests parse the published hex strings and compare against live computation, ensuring the specification and implementation remain in agreement.

---

16. Empirical Verification Suite

This section documents the empirical test results that validate the theoretical security claims.

16.1 Non-Reconstructibility Verification

The `examples/proof.rs` program generates 10 ciphertexts from the same plaintext and key, measuring output randomness:

Metric	Measured Value	Expected
Shannon entropy per byte	2.322 bits	High (random distribution)
Chi-squared statistic (256 bins)	251.00	~256 (uniform)
Mean Hamming distance between pairs	122/256 bits (47.7%)	~128/256 (50%)
Unique ciphertexts out of 10	10/10	10/10
Unique entropy snapshots	10/10	10/10
Mean pairwise bit difference	49.6%	~50%

The near-ideal 50% pairwise bit difference confirms that successive ciphertexts are cryptographically independent despite identical input.

→ `examples/proof.rs`

16.2 QKD BB84 Simulation Verification

The `examples/qkd_demo.rs` program simulates full BB84 quantum key distribution:

Scenario	Qubits	Sifted	QBER	Result
Clean channel (no Eve)	4,096	~1,970	0.0%	SUCCESS: shared key derived
Eve intercept-resend	4,096	~2,072	~24.5%	DETECTED and ABORTED

In the eavesdropper scenario, Eve's intercept-resend attack introduces ~25% QBER (theoretical expectation for BB84), correctly exceeding the 11% threshold. Eve's guess accuracy is ~50% (2,072/4,096), confirming that interception provides no usable information.

→ `examples/qkd_demo.rs` , `src/qkd.rs`

16.3 Branch Number Measurement

The `examples/differential.rs` program measures the branch number of the KK permutation:

Metric	Value
Minimum active words per differential	2
Average branch number (over 50,000 random inputs)	2.98 out of 5
Round at which full diffusion is achieved	Round 4
Safety margin	8x (32 rounds used, 4 needed for full diffusion)

A minimum branch number of 2 ensures that every non-zero input difference activates at least 2 output words, providing guaranteed diffusion through the quintet structure.

→ `examples/differential.rs`

16.4 Per-Position Ciphertext Independence

The `tests/integration.rs` per-position independence test encrypts a 256-byte plaintext where every byte is identical (`0xAA`), then counts unique byte values at each position across multiple encryptions:

Metric	Value
Plaintext	256 bytes, all `0xAA`
Unique ciphertext byte values per position	> 50 (out of 256 possible)
Result	PASS: no positional bias detected

This confirms that the stream cipher produces position-independent keystream. Identical plaintext bytes at different offsets encrypt to statistically independent ciphertext bytes.

→ `tests/integration.rs` (per-position independence test)

---

17. Continuous Fuzzing Infrastructure

KK-Crypto maintains 8 independent fuzz targets under `fuzz/fuzz_targets/` , built with `cargo-fuzz` (libFuzzer backend):

Fuzz Target	Module	Property Tested
`hash_fuzz`	`kk_mix`	Hash never panics on arbitrary input
`kdf_fuzz`	`kk_mix`	KDF handles arbitrary key, salt, info, output length
`mac_fuzz`	`kk_mix`	MAC + verify roundtrip on arbitrary input
`roundtrip_fuzz`	`codec`	Encode/decode roundtrip: output equals input for arbitrary plaintext
`aead_fuzz`	`codec`	AEAD encode/decode roundtrip with arbitrary plaintext and AAD

`session_fuzz`	`session`	Rope Ratchet encode/decode roundtrip preserves plaintext
`temporal_fuzz`	`temporal`	Temporal commit/verify roundtrip on arbitrary ciphertext
`eka_fuzz`	`eka`	Full EKA 3-message handshake completes without panic

All fuzz targets use `arbitrary::Unstructured` to generate valid inputs from raw fuzzer bytes, ensuring coverage of edge cases (empty inputs, maximum-length inputs, adversarial byte patterns).

Running:

```
cargo +nightly fuzz run <target> -- -max_total_time=300
```

---

18. Parallel Merkle Trunk Tamper Detection

18.1 Construction

`KkParallelPacket` extends the standard `KkPacket` with a Merkle tree root over the ciphertext chunks:

$$\text{leaf}_i = \text{kk_hash}(\text{chunk}_i)$$ $$\text{root} = \text{kk_hash}(\text{leaf}_0 ,|, \text{leaf}*1 ,|, \cdots ,|, \text{leaf}*{n-1})$$

The Merkle root is stored alongside the temporal commitment. On decode, the root is recomputed and compared; any mismatch produces an immediate `MerkleMismatch` error.

→ `codec::KkParallelPacket` , `codec::compute_merkle_root()`

18.2 Tamper Detection Tests

Test	Description	Result
Flip single bit in ciphertext	Modify one bit in chunk 0	`MerkleMismatch` error
Swap two chunks	Exchange chunk 0 and chunk 1	`MerkleMismatch` error
Truncate ciphertext	Remove last chunk	`MerkleMismatch` error
Legitimate parallel decode	Unmodified packet	SUCCESS: plaintext recovered

The Merkle trunk enables parallel integrity verification: each chunk can be verified independently before decryption proceeds.

→ `codec::encode_parallel()` , `codec::decode_parallel()` , `tests/integration.rs`

End of specification.

12. KK-EKA (Entropy Key Agreement)

12.1 Overview

KK-EKA is a 3-message PSK-based key agreement protocol where both parties contribute fresh entropy. No public-key cryptography - authentication is via KK-MAC over a pre-shared key.

12.2 Protocol Flow

```
Alice (Initiator)                                  Bob (Responder)
─────────────────                                  ───────────────

ε_a ← gather()
commit_a ← KK-Hash(serialize(ε_a))

    ─── msg1: commit_a (32B) ──────────────────►
                                                    ε_b ← gather()
                                                    auth_b ← KK-MAC(psk, serialize(ε_b) ‖
commit_a)

    ◄─── msg2: serialize(ε_b) (48B) + auth_b (32B) ───

verify auth_b
auth_a ← KK-MAC(psk, serialize(ε_a) ‖ serialize(ε_b))

    ─── msg3: serialize(ε_a) (48B) + auth_a (32B) ──►
                                                    verify: KK-Hash(serialize(ε_a)) = commit_a
                                                    verify auth_a

BOTH: session_key ← KK-KDF(psk, serialize(ε_a) ‖ serialize(ε_b), "KK-EKA-session", 32)
BOTH: zeroize ephemeral state
```

12.3 Wire Formats

Message	Size	Contents
`EkaMsg1`	32 bytes	`commit_a` (hash of Alice's serialized entropy)
`EkaMsg2`	80 bytes	`entropy_b_bytes` (48B) ‖ `auth_b` (32B)
`EkaMsg3`	80 bytes	`entropy_a_bytes` (48B) ‖ `auth_a` (32B)

12.4 Initiator (Alice)

12.4.1 New

1. $\varepsilon_a \leftarrow \text{entropy::gather()}$
2. $\text{commit}_a \leftarrow \text{KK-Hash}(\varepsilon_a.\text{to_bytes()})$
3. Send `EkaMsg1` ${\text{commit}_a}$
4. Retain state: $(\text{psk}, \varepsilon_a, \text{commit}_a)$

→ `eka::EkaInitiator::new()`

12.4.2 Process Message 2

Input: `EkaMsg2` ${\varepsilon_{b}^{\text{bytes}}, \text{auth}_b}$.

1. **Verify Bob's MAC:**
   - $\text{msg} \leftarrow \varepsilon_{b}^{\text{bytes}} \parallel \text{commit}_a$
   - `kk_mac_verify(psk, msg, auth_b)` → `CommitmentMismatch` if false
2. **Compute auth_a:**

- $\text{msg} \leftarrow \varepsilon_a.\text{to_bytes()} \parallel \varepsilon_{b}^{\text{bytes}}$
- $\text{auth}_a \leftarrow \text{KK-MAC}(\text{psk}, \text{msg})$

3. **Derive session key:**
   - $\sigma \leftarrow \varepsilon_a.\text{to_bytes()} \parallel \varepsilon_{b}^{\text{bytes}}$ (96-byte salt)
   - $\text{session_key} \leftarrow \text{KK-KDF}(\text{psk}, \sigma, \texttt{b"KK-EKA-session"}, 32)$
4. Return `(EkaMsg3, session_key)`. Zeroize initiator state on drop.

→ `eka::EkaInitiator::process_msg2()`

12.5 Responder (Bob)

12.5.1 New

Input: PSK, `EkaMsg1` ${\text{commit}_a}$.

1. $\varepsilon_b \leftarrow \text{entropy::gather()}$
2. $\text{msg} \leftarrow \varepsilon_b.\text{to_bytes()} \parallel \text{commit}_a$
3. $\text{auth}_b \leftarrow \text{KK-MAC}(\text{psk}, \text{msg})$
4. Send `EkaMsg2` ${\varepsilon_b.\text{to_bytes()}, \text{auth}_b}$
5. Retain state: $(\text{psk}, \varepsilon_b.\text{to_bytes()}, \text{commit}_a)$

→ `eka::EkaResponder::new()`

12.5.2 Process Message 3

Input: `EkaMsg3` ${\varepsilon_{a}^{\text{bytes}}, \text{auth}_a}$.

1. **Verify commitment:**
   - $\text{KK-Hash}(\varepsilon_{a}^{\text{bytes}}) = \text{commit}_a$ → `CommitmentMismatch` if false
2. **Verify Alice's MAC:**
   - $\text{msg} \leftarrow \varepsilon_{a}^{\text{bytes}} \parallel \varepsilon_b.\text{to_bytes()}$
   - `kk_mac_verify(psk, msg, auth_a)` → `CommitmentMismatch` if false
3. **Derive session key:**
   - $\sigma \leftarrow \varepsilon_{a}^{\text{bytes}} \parallel \varepsilon_b.\text{to_bytes()}$ (96-byte salt)
   - $\text{session_key} \leftarrow \text{KK-KDF}(\text{psk}, \sigma, \texttt{b"KK-EKA-session"}, 32)$
4. Return session_key. Zeroize responder state on drop.

→ `eka::EkaResponder::process_msg3()`

---

13. Security Claims

13.1 Collision Resistance (KK-Hash)

Claim: KK-Hash provides 2^{128} collision resistance (birthday bound on 256-bit output).

Basis: The sponge capacity of 384 bits prevents internal state collisions with probability $> 1 - 2^{-192}$. The output is 256 bits, so the birthday bound governs the external collision probability at 2^{128}.

13.2 Preimage Resistance (KK-Hash)

Claim: KK-Hash provides 2^{192} preimage resistance (capacity-limited).

Basis: Inverting the sponge requires guessing the 384-bit capacity, providing 2^{192} single-target preimage resistance.

13.3 KDF Security

Claim: KK-KDF is a PRF (pseudorandom function) under the assumption that the KK permutation is a pseudorandom permutation (PRP).

Basis: The sponge-based KDF with domain separation, length-prefixed inputs, and capacity isolation follows the standard sponge-PRF model. Additionally, KK-KDF uses entropy-derived rotations from the salt, making the permutation structure itself key-dependent.

13.4 MAC Unforgeability

Claim: KK-MAC provides 2^{128} existential unforgeability under chosen-message attack (EUF-CMA), assuming the KK permutation is a PRP.

Basis: The keyed sponge MAC with domain separation follows the standard sponge-MAC security model. The length-prefixed key prevents length-extension attacks. The 384-bit capacity provides 2^{192} state-recovery resistance, but the 256-bit tag limits forgery to 2^{-256} per attempt.

13.5 Forward Secrecy (Rope Ratchet)

Claim: The Rope Ratchet provides ~192-bit forward secrecy.

Basis: Compromise of the current ratchet state reveals the current chain strand (32B) but the previous chain strand was overwritten and zeroized. Recovering it requires inverting KK-KDF, which requires guessing the 384-bit sponge capacity. The 4-strand mixing through entropy-derived rotations further strengthens the claim: to recover a past message key, an attacker would need to invert a sponge whose algebraic structure (rotation schedule) is unknown.

13.6 Contributory Key Agreement (KK-EKA)

Claim: KK-EKA provides a contributory key agreement: neither party alone controls the session key.

Basis:

- The session key is $\text{KK-KDF}(\text{psk}, \varepsilon_a \parallel \varepsilon_b, \text{info}, 32)$, depending on both parties' entropy
- Alice commits to ε_a before seeing ε_b (hash commitment in msg1)
- Bob's entropy is revealed before Alice's, but Alice cannot change ε_a after commitment
- Both parties authenticate via KK-MAC over the PSK, preventing impostor contributions

13.7 Temporal Binding

Claim: The temporal commitment binds the ciphertext to the entropy snapshot at the moment of creation. Modifying the ciphertext, snapshot, or either party's secret invalidates the commitment.

Basis: The commitment MAC covers $\varepsilon.\text{bytes} \parallel \varepsilon.\text{timestamp} \parallel C$, and the commitment key is derived from the shared secret and entropy. Forging requires knowledge of the shared secret.

13.8 DDR Differential Resistance

Claim: DDR prevents efficient differential cryptanalysis by forcing exponential path explosion.

Basis: Any differential trail through DDR must account for all 64 possible rotation distances simultaneously (since the rotation depends on the data difference itself). Standard differential analysis tools track fixed rotations; DDR invalidates this assumption. Additionally, the constant-time implementation prevents timing-based distinguishers.

13.9 Limitations

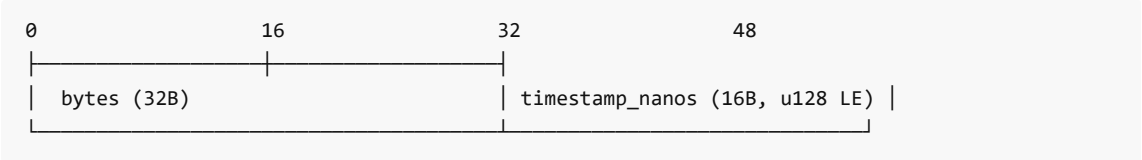
- KK is a novel, un-audited primitive. It has **not** been reviewed by third-party cryptographers. It should not be used for production security until independent analysis is complete.
- The base codec (without Rope Ratchet) has no forward secrecy.
- Replay protection is not built into the base codec; callers must add sequence numbers or use the bound commitment protocol.
- Side-channel hardening is limited to zeroization of intermediate keys and constant-time MAC comparison. Variable-time modular multiplication (MFR) may leak information on some microarchitectures.

---

14. Wire Format Diagrams

All multi-byte integers are little-endian. All lengths are in bytes.

14.1 EntropySnapshot (48 bytes)



Offset	Size	Field
0	32	bytes (entropy)
32	16	timestamp_nanos (u128 LE)
Total:	48	

14.2 TemporalCommitment (32 bytes)

Offset	Size	Field
0	32	mac (KK-MAC tag)
Total:	32	

14.3 TemporalProof (96 bytes)

Offset	Size	Field
0	32	mac (KK-MAC-Entropy tag)
32	32	nonce (challenge)

```
64      32    prev_mac (chain link)
──────  ────
Total:  96
```

14.4 KkPacket

```
Offset       Size            Field
──────       ────            ──────────────────────
0            4               ct_len (u32 LE)
4            ct_len          ciphertext
4+ct_len     48              EntropySnapshot
4+ct_len+48  32              TemporalCommitment (mac)
──────       ────
Total:       4 + ct_len + 48 + 32 = ct_len + 84
```

14.5 KkSealedMessage (Split-Channel)

```
Offset       Size            Field
──────       ────            ──────────────────────
0            4               ct_len (u32 LE)
4            ct_len          ciphertext
4+ct_len     32              TemporalCommitment (mac)
──────       ────
Total:       4 + ct_len + 32 = ct_len + 36
```

14.6 KkBoundPacket

```
Offset       Size            Field
──────       ────            ──────────────────────
0            4               ct_len (u32 LE)
4            ct_len          ciphertext
4+ct_len     48              EntropySnapshot
4+ct_len+48  96              TemporalProof (mac + nonce + prev_mac)
──────       ────
Total:       4 + ct_len + 48 + 96 = ct_len + 148
```

14.7 KkAeadPacket

```
Offset              Size       Field
──────              ────       ──────────────────────
0                   4          aad_len (u32 LE)
4                   aad_len    associated data (plaintext)
4+aad_len           4          ct_len (u32 LE)
4+aad_len+4         ct_len     ciphertext
4+aad_len+4+ct_len  48         EntropySnapshot
...+48              32         TemporalCommitment (mac)
──────              ────
Total:              8 + aad_len + ct_len + 48 + 32 = aad_len + ct_len + 88
```

14.8 RopeStep (56 bytes)

```
Offset  Size  Field
──────  ────  ──────────────────────
0       8     counter (u64 LE)
8       48    EntropySnapshot
──────  ────
Total:  56
```

14.9 RopePacket

```
Offset   Size            Field
──────   ────            ──────────────────────
0        56              RopeStep (counter + snapshot)
56       variable        KkPacket (inner encrypted payload)
──────   ────
Total:   56 + (ct_len + 84) = ct_len + 140
```

14.10 RopeAeadPacket

```
Offset   Size            Field
──────   ────            ──────────────────────
0        56              RopeStep (counter + snapshot)
56       variable        KkAeadPacket (inner AEAD payload)
──────   ────
Total:   56 + (aad_len + ct_len + 88) = aad_len + ct_len + 144
```

14.11 EKA Messages

```
EkaMsg1 (32 bytes):
Offset  Size  Field
──────  ────  ──────────────────────
0       32    commit_a (KK-Hash of serialized ε_a)

EkaMsg2 (80 bytes):
Offset  Size  Field
──────  ────  ──────────────────────
0       48    entropy_b_bytes (serialized ε_b)
48      32    auth_b (KK-MAC tag)

EkaMsg3 (80 bytes):
Offset  Size  Field
──────  ────  ──────────────────────
0       48    entropy_a_bytes (serialized ε_a)
48      32    auth_a (KK-MAC tag)
```

---

15. Test Vector References

Deterministic test vectors are defined in `KK_TEST_VECTORS.md` and verified by the `tests/integration.rs` test suite (44 vector tests). All vectors use fixed entropy snapshots and timestamps to ensure reproducibility.

15.1 Vector Categories

Category	Count	Description
Basic encode/decode	6	Roundtrip with various plaintext sizes
AEAD encode/decode	6	Roundtrip with various AAD and plaintext sizes
Deterministic ciphertext	8	Exact ciphertext bytes for fixed inputs
KK-Hash	4	Exact digest for known inputs
KK-KDF	4	Exact derived key for known inputs
KK-MAC	4	Exact tag for known key/message
Session (Rope Ratchet)	6	Sequential encode/decode with ratchet state
EKA key agreement	6	Full protocol transcript with known entropy

15.2 Reference File

See `KK_TEST_VECTORS.md` in the repository root for:

- All input values (shared secrets, plaintexts, AAD, entropy snapshots)
- Expected output values (ciphertexts, commitments, MACs, derived keys)
- Step-by-step intermediate values for hand verification

15.3 Running Vectors

```
cargo test                          # All 170 tests including 44 vector tests
cargo test --test integration       # Integration tests only
cargo test vector                   # Filter for vector-specific tests
```

Appendix A. Module Structure

```
src/
├── lib.rs          - Module declarations, re-exports, crate documentation
├── kk_mix.rs       - KK permutation, MFR, DDR, sponge, hash, KDF, MAC
├── kk_mix_avx512.rs - AVX-512 vectorized permutation (x86_64 only)
├── entropy.rs      - Entropy sources, gathering, snapshot
├── kdf.rs          - Per-chunk key derivation, commitment key derivation
├── codec.rs        - Stream cipher, packet formats, encode/decode
├── temporal.rs     - Temporal commitment, bound proofs
├── session.rs      - Rope Ratchet, forward-secret session API
├── eka.rs          - Entropy Key Agreement protocol
├── qkd.rs          - Quantum Key Distribution simulation
└── error.rs        - Error types
```

Appendix B. Code ↔ Spec Cross-Reference

Spec Section	Function	Source File	Line
§3.1 MFR	`mfr()`	`kk_mix.rs`	180
§3.2 DDR	`ddr()`	`kk_mix.rs`	209
§3.3 QuintetRound	`quintet_round()`	`kk_mix.rs`	254
§4 Permutation	`kk_permute_n()`	`kk_mix.rs`	279
§5.2 Rotation derivation	`rotations_from_entropy()`	`kk_mix.rs`	366
§5.3 Entropy mixing	`kk_entropy_mix()`	`kk_mix.rs`	815
§6.3 Absorb	`KkSponge::absorb()`	`kk_mix.rs`	462
§6.4 Finalize	`KkSponge::finalize_absorb()`	`kk_mix.rs`	506
§6.5 Squeeze	`KkSponge::squeeze()`	`kk_mix.rs`	519
§7.1 Hash	`kk_hash()`	`kk_mix.rs`	567
§7.2 KDF	`kk_kdf()`	`kk_mix.rs`	593
§7.3 KDF Batch	`kk_kdf_batch_8()`	`kk_mix.rs`	626
§7.4 MAC	`kk_mac()`	`kk_mix.rs`	729
§7.5 MAC Verify	`kk_mac_verify()`	`kk_mix.rs`	748
§7.6 MAC Entropy	`kk_mac_with_entropy()`	`kk_mix.rs`	763
§8.1 Chunk KDF	`derive_symbol_key()`	`kdf.rs`	36
§8.2 Batch KDF	`derive_symbol_key_batch()`	`kdf.rs`	67
§8.3 Keystream XOR	`xor_with_keystream()`	`codec.rs`	655
§8.4 Encode	`encode()`	`codec.rs`	195
§8.5 Decode	`decode()`	`codec.rs`	224
§9.1 Commit key	`derive_commitment_key()`	`kdf.rs`	55
§9.2 Commit	`commit()`	`temporal.rs`	89
§9.3 Verify	`verify()`	`temporal.rs`	108
§9.4.2 Commit bound	`commit_bound()`	`temporal.rs`	293
§9.4.3 Verify bound	`verify_bound()`	`temporal.rs`	339
§10.2 AEAD commit	`commit_aead()`	`temporal.rs`	142
§10.3 AEAD encode	`encode_aead()`	`codec.rs`	557

§10.4 AEAD decode	`decode_aead()`	`codec.rs`	582
§11.2 Ratchet init	`RopeRatchet::new()`	`session.rs`	185
§11.3 Ratchet step	`RopeRatchet::step()`	`session.rs`	288
§11.7 Encode session	`encode_session()`	`session.rs`	444
§11.8 Decode session	`decode_session()`	`session.rs`	469
§12.4 EKA Initiator	`EkaInitiator`	`eka.rs`	151
§12.5 EKA Responder	`EkaResponder`	`eka.rs`	244

End of specification.

John A Keeney Entrouter 2026 hello@entrouter.com